C12 Input to Blend Tree

CRE132 worksheet

What you'll make

A side-view hero who stands idle, walks with A/D or the arrows, and runs while you hold Shift, with the animation following the speed. On a gamepad, pushing the stick part of the way makes the hero walk slowly, and the walk animation slows down to match.

In C07 each animation was a separate state with transitions between them. This time you'll use a **blend tree**: one state that picks the right animation from a number, **Speed**. You'll use the **Run** action in C11's input asset, write a movement script that walks and runs, and an animator script that sends the hero's real speed to the blend tree.

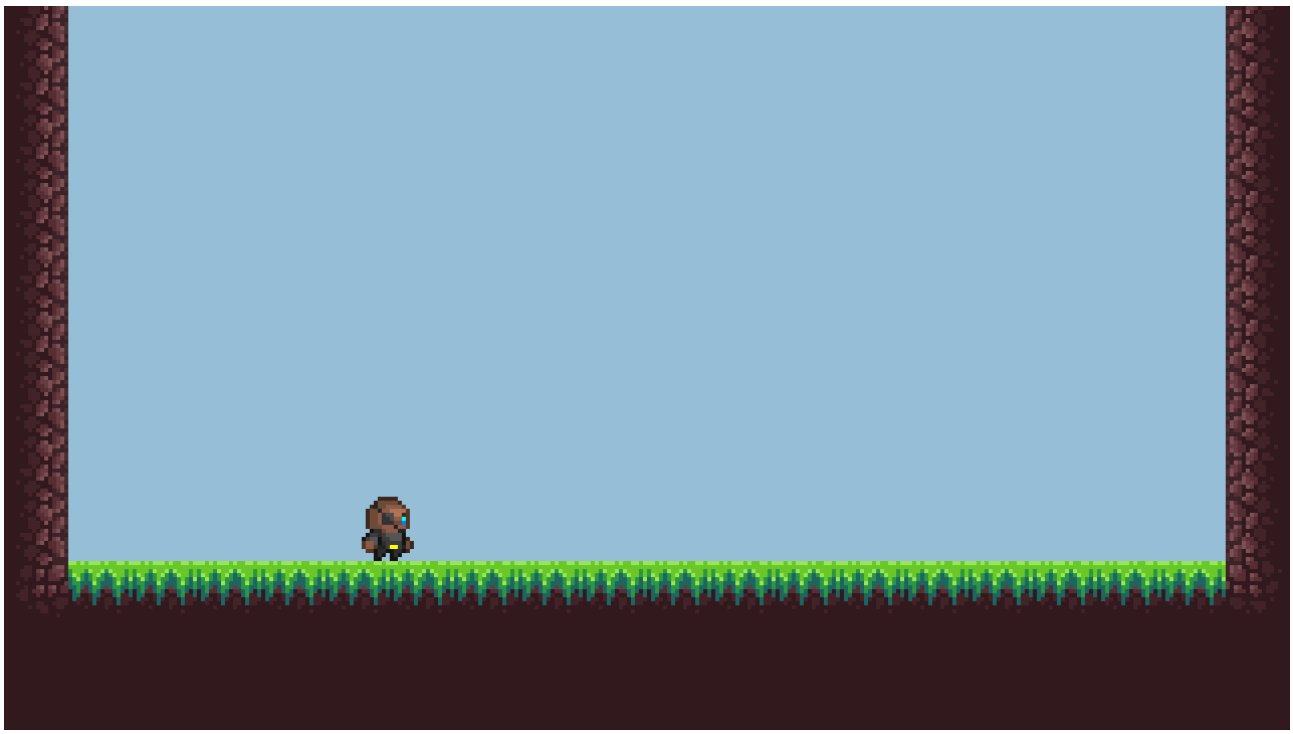


Figure 1: The finished scene: the hero on a long grassy floor between two cliffs.

Before you start

1. Unzip `CRE132_C12_InputToBlendTree_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_C12_InputToBlendTree_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Core/C12_InputToBlendTree`.
3. Press **Play**. The **Player** stands and breathes (its idle animation) but can't move yet. Press Play again to stop.

The Player already has a **Rigidbody 2D**, a **Capsule Collider 2D** and an **Animator** using `Animation/Controllers/Hero`. The clips are `Hero_Idle` and `Hero_Walk` in `Animation/Clips/ElvGames`. There's no run sheet in the art pack.

C11's input asset is already in the project: `Assets/_game/Input/CRE132Controls`.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The Run action

Double-click `CRE132Controls` to open it. The **Player** map has **Move** and **Jump** from C11. Add a third action the same way:

1. Click **+** next to **Actions** and name it `Run`. Leave its **Action Type** as **Button**: it's either held or not.
2. Select the binding under Run (**<No Binding>**), click the **Path** drop-down, press **Listen** and press **Left Shift**.
3. Click **+** on Run, choose **Add Binding**, and set its Path to **Button West [Gamepad]** (X on Xbox, Square on PlayStation).

Click **Save Asset** at the top of the window (or tick **Auto-Save**) and close the editor. `Player/Run` now shows inside `CRE132Controls` in the Project window.

2. Walking and running: RunningPlayerMovement

Make a folder `Assets/_game/Scripts/C12_InputToBlendTree` and create `RunningPlayerMovement` in it:

```
// Moves a side-view player with the Input System: walk normally, run while the Run
// button is held.
// The animation is handled by a separate script (HeroAnimator), so this one only
// deals with movement.
using UnityEngine;
using UnityEngine.InputSystem;

public class RunningPlayerMovement : MonoBehaviour
{
    [SerializeField] private float walkSpeed = 3f;
    [SerializeField] private float runSpeed = 6f;

    // Drag Player/Move and Player/Run from the CRE132Controls asset into these slots.
    [SerializeField] private InputActionReference moveAction;
    [SerializeField] private InputActionReference runAction;

    private Rigidbody2D body;
    private float horizontalInput;
    private bool isRunning;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
    }

    private void OnEnable()
    {
        moveAction.action.Enable();
        runAction.action.Enable();
    }

    private void OnDisable()
    {
        moveAction.action.Disable();
        runAction.action.Disable();
    }
```

```
    private void Update()
    {
        Vector2 moveValue = moveAction.action.ReadValue<Vector2>();
        horizontalInput = moveValue.x;

        // IsPressed is true for every frame the button is held down (like
        // Input.GetKey).
        isRunning = runAction.action.IsPressed();
    }

    private void FixedUpdate()
    {
        float speed = walkSpeed;
        if (isRunning)
        {
            speed = runSpeed;
        }

        // A gamepad stick pushed halfway gives 0.5, so the player moves at half speed.
        body.linearVelocity = new Vector2(horizontalInput * speed,
            body.linearVelocity.y);
    }
}
```

- This is C11's **InputSystemPlayerMovement** with two speeds instead of one.
- **`IsPressed()`** is true on every frame the button is held, like `Input.GetKey`. (`WasPressedThisFrame()`, used for Jump, is only the first frame.)
- **Both actions are enabled** in `OnEnable`: an action you forget to enable never does anything.

Add it: select the **Player**, **Add Component > RunningPlayerMovement**. Leave **Walk Speed** at **3** and **Run Speed** at **6**. Set **Move Action** to **Player/Move** and **Run Action** to **Player/Run**.

Test it: press Play. A/D walks and holding Shift runs, but the hero slides along still breathing: the animation doesn't know about the speed yet.

3. A blend tree

The Starter's `Hero` controller already holds the finished blend tree. You'll build your own to see how.

1. Make a folder `Assets/_game/Animation/Mine`. In it, **Create > Animation > Animator Controller** and name it `MyHero`. Drag it into the **Controller** slot of the Player's **Animator**.
2. Open **Window > Animation > Animator**. On the **Parameters** tab, add a **Float** called `Speed`.
3. Right-click the empty grid and choose **Create State > From New Blend Tree**. Select the new state and rename it **Locomotion** in the Inspector. It's orange: the default state.
4. Double-click **Locomotion** to open the blend tree. Click the **Blend Tree** node. In the Inspector, set **Blend Type** to **1D** and **Parameter** to **Speed**.
5. Under **Motion**, click **+ > Add Motion Field** three times. Drag `Hero_Idle` into the first, and `Hero_Walk` into the second *and* the third.
6. Untick **Automate Thresholds**, then type the thresholds **0**, **3** and **6**: the real speeds in units per second, matching Walk Speed and Run Speed.
7. In the third row, set the speed (the column with the clock icon) to **1.8**. That's the run: there's no run sheet, so it's the walk played 1.8 times faster.

Try the preview: at the bottom of the Inspector, drag the **Speed** slider (the red line on the graph). The preview switches from idle to walk to the fast walk.

- **One state replaces several.** In C07, adding Run would have meant a third state and more transitions, each with a condition. Here it's one more row.

- **Sprite animation can’t mix two pictures.** The tree plays the motion with the most weight and blends the playback **speed**, which is why a half-pushed stick gives a slower walk.

Go back to the Base Layer (click **Base Layer** at the top of the Animator window).

4. Telling the blend tree: HeroAnimator

Create `HeroAnimator` in `Scripts/C12_InputToBlendTree`:

```
// Sends the player's speed to the Animator's blend tree and turns the sprite to face
// the way it moves.
// It reads the Rigidbody2D's velocity, so it works with any movement script.
using UnityEngine;

public class HeroAnimator : MonoBehaviour
{
    // Below this speed the player counts as standing still, so it keeps facing the
    // same way.
    [SerializeField] private float turnThreshold = 0.1f;

    private Rigidbody2D body;
    private Animator animator;
    private SpriteRenderer spriteRenderer;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        animator = GetComponent<Animator>();
        spriteRenderer = GetComponent<SpriteRenderer>();
    }

    private void Update()
    {
        float horizontalVelocity = body.linearVelocity.x;

        // The blend tree only cares how fast, not which way, so send a positive
        // number.
        float speed = Mathf.Abs(horizontalVelocity);
        animator.SetFloat("Speed", speed);

        // The hero's sprites face right. Flip them when moving left.
        if (horizontalVelocity < -turnThreshold)
        {
            spriteRenderer.flipX = true;
        }
        else if (horizontalVelocity > turnThreshold)
        {
            spriteRenderer.flipX = false;
        }
    }
}
```

- **It reads the velocity, not the input.** Send the Animator what actually happened: a player pushing against a wall isn’t moving, so it stays idle instead of walking on the spot. It also means this script works with any movement script.
- **`Mathf.Abs`** turns −3 (walking left) into 3. The blend tree only cares how fast; `flipX` takes care of which way.
- **`turnThreshold`:** the hero only turns when moving faster than 0.1. Without it, a tiny velocity on the frame it stops could flip it back the wrong way.

Add it: on the **Player**, **Add Component > HeroAnimator**. Leave **Turn Threshold** at **0.1**.

Test it: put the Animator window beside the Game view, select the Player and press Play. Walk and run: the Speed value and the blend tree's bar move as you go, and the hero faces the way it moves and keeps facing that way when it stops.

Check your work

- ☐ `CRE132Controls` has a **Run** action bound to Left Shift and Button West.
- ☐ Standing still is idle; A/D walks at 3; holding Shift runs at 6 with a faster walk.
- ☐ The hero faces left when walking left, and keeps facing left after stopping.
- ☐ Walking into a cliff wall shows idle, not a walk on the spot.
- ☐ With a gamepad, a half-pushed stick gives a slower walk.
- ☐ Compare with `CRE132_C12_InputToBlendTree_Finished` if something behaves differently.

Key points

- **A blend tree replaces a group of similar states** with one state driven by a number.
- **With sprite animation, blending can't mix two pictures.** The tree picks the motion with the most weight and blends the playback speed.
- **Send the Animator what actually happened (velocity), not what was pressed.**
- **Facing direction lives in the animator script.** Only flip when moving faster than a small threshold.

If something goes wrong

- **NullReferenceException in OnEnable:** the **Move Action** or **Run Action** slot is empty.
- **"Parameter 'Speed' does not exist":** the parameter in `MyHero` is spelt differently.
- **Running shows the normal walk:** the third threshold isn't 6, or its speed isn't 1.8. Check **Automate Thresholds** is unticked; ticking it spreads the thresholds out by itself.
- **The hero walks on the spot against a wall:** you're sending the input to the Animator rather than the velocity. Check `body.linearVelocity.x`.
- **The hero never animates:** the Player's Animator has no controller, or Locomotion isn't the default (orange) state.

Try it

1. Raise **Run Speed** to 8 and set the third threshold to 8 too. What happens if you forget the threshold?
2. Smooth the value: `animator.SetFloat("Speed", speed, 0.1f, Time.deltaTime)`. The extra two numbers are damping.
3. Add C11's **InputSystemJump** to the Player (copy the script across from your C11 project). What animation plays in the air? P06 adds jump and fall states.